



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Конструктор из геометрических примитивов»

Студент ИУ7-53Б
(Группа)

(Подпись, дата)

М.А. Семенчук
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Д.А. Шибанова
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ	4
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	5
ВВЕДЕНИЕ	6
1 Аналитический раздел	7
1.1 Трассировка лучей	7
1.1.1 Алгоритм прямой трассировки лучей	7
1.1.2 Алгоритм обратной трассировки лучей	8
1.1.3 Выбор алгоритма	9
1.2 Анализ моделей освещения	9
1.2.1 Модель освещения Ламберта	10
1.2.2 Модель освещения Фонга	12
1.2.3 Модель освещения Блинна-Фонга	14
1.2.4 Выбор модели освещения	15
1.3 Обзор геометрических примитивов	15
1.3.1 Сфера	15
1.3.2 Плоскость	15
1.3.3 Четырехугольник	15
1.3.4 Треугольник	15
1.3.5 Параллелепипед, правильная призма и пирамида . .	15
2 Конструкторский раздел	16
2.1 Алгоритм построения изображений	16
2.2 Алгоритм обратной трассировки лучей	18
3 Технологический раздел	20
3.1 Требования к ПО	20
3.2 Средства реализации	20
3.3 Диаграмма классов	21
3.4 Типы и структуры данных	21
3.5 Алгоритмы	21
3.6 Графический интерфейс пользователя	22

4 Исследовательский раздел	23
4.1 Анализ производительности ПО	23
ЗАКЛЮЧЕНИЕ	24
ПРИЛОЖЕНИЕ А	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ОПРЕДЕЛЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие термины с соответствующими определениями.

Рендеринг —

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие сокращения и обозначения.

GI — global illumination (глобальное освещение)

ПО — программное обеспечение

ЦП (CPU) — центральный процессор (central proccesing unit)

ОЗУ (RAM) — оперативное запоминающее устройство (random access memory)

ГП (GPU) — графический процессор (graphics proccesing unit)

—

SFML — Simple and Fast Multimedia Library

GLFW — Graphics Library Framework

SDL — Simple DirectMedia Layer

ВВЕДЕНИЕ

Компьютерная графика стала неотъемлемой частью современного мира и с каждым годом находит свое применения все в новых аспектах жизни. Она применяется в кинематографе, игровой индустрии, архитектуре, медицине и других областях.

Алгоритм трассировка лучей в последние годы активно применяется в компьютерной графике. В связи с этим производители графических карт (GPU) начали добавлять в свои устройства ядра, специализирующиеся на трассировке лучей (так называемая RT-ядра). В стороне не остались разработчики графических API и библиотек, которые активно добавляют в свои продукты поддержку этой технологии. Известными примерами могут стать DirectX Ray Tracing (DXR), Vulkan RTX, CUDA/OptiX и другие. Многосторонняя поддержка этой технологии позволяет создавать все более фотореалистичные изображения с большим разрешением и лучшим качеством.

1 Аналитический раздел

1.1 Трассировка лучей

Трассировка лучей (от англ. Ray tracing) — это метод рендеринга, который позволяет реалистично имитировать освещение среды за счет использования физически корректной модели поведения света. Распространяясь по сцене и взаимодействуя с ее объектами, свет может отражаться, преломляться, поглощаться и рассеиваться.

1.1.1 Алгоритм прямой трассировки лучей

Главная идея, лежащая в основе алгоритма прямой трассировки лучей, заключается в том, что наблюдатель видит объекты посредством испускаемого источниками света, который многократно отражаясь, согласно законам оптики, доходит до глаз наблюдателя. Метод прямой трассировки предполагает построения траекторий лучей от всех источников освещения ко всем точкам всех объектов сцены, лучи отражаются, преломляются или проходят сквозь него и в результате достигает наблюдателя. Если объект не является отражающим или прозрачным, то траектория луча на этой точке обрывается.

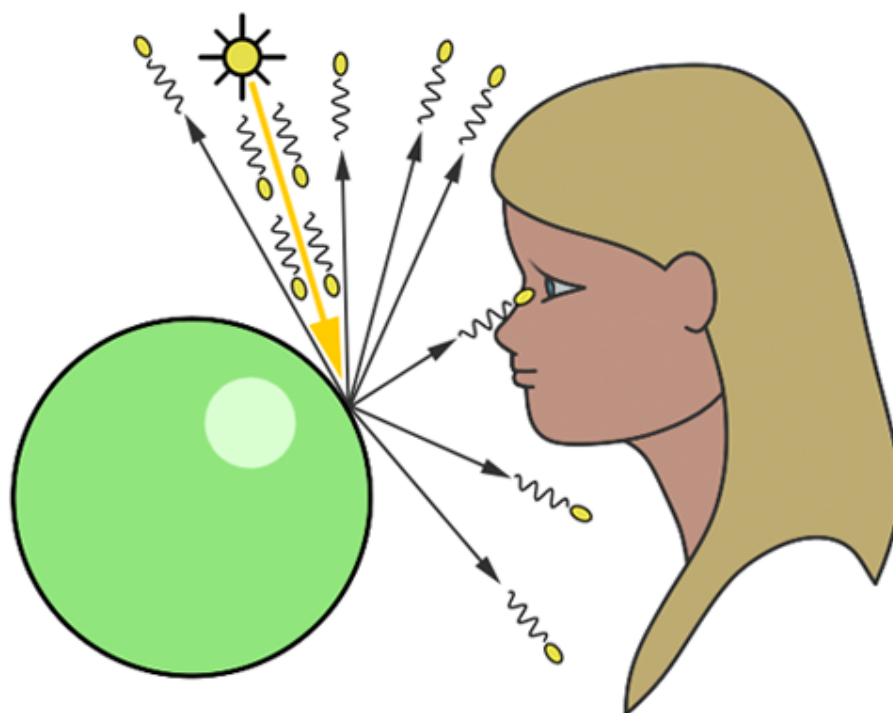


Рисунок 1.1 – Алгоритм прямой трассировки лучей

Основным недостатком этого алгоритма является излишне большое число рассматриваемых лучей, приводящее с существенным затратам вычислительных мощностей, так как лишь малая часть лучей достигает точки наблюдения. Однако данный алгоритм имитирует реальное поведение света и применяется в задачах, в которых точность важнее скорости (оптика и фотоника, расчет освещенности помещения и тд).

1.1.2 Алгоритм обратной трассировки лучей

На отображение сцены влияют только лучи, достигшие наблюдателя. Благодаря тому, что свет имеет особенность распространяться одинаково в обоих направлениях, можно искусственно поменять направления лучей на противоположное, и испускать лучи из точки наблюдения. Это позволяет рассматривать исключительно лучи, которые гарантированно попадут в камеру. Данный метод, основанный на испускании лучей от камеры к объектам сцены, называется обратной трассировкой лучей. Данный подход позволяет существенно сократить количество обрабатываемых лучей и обеспечить более эффективное формирование изображений при приемлемой степени физической достоверности.

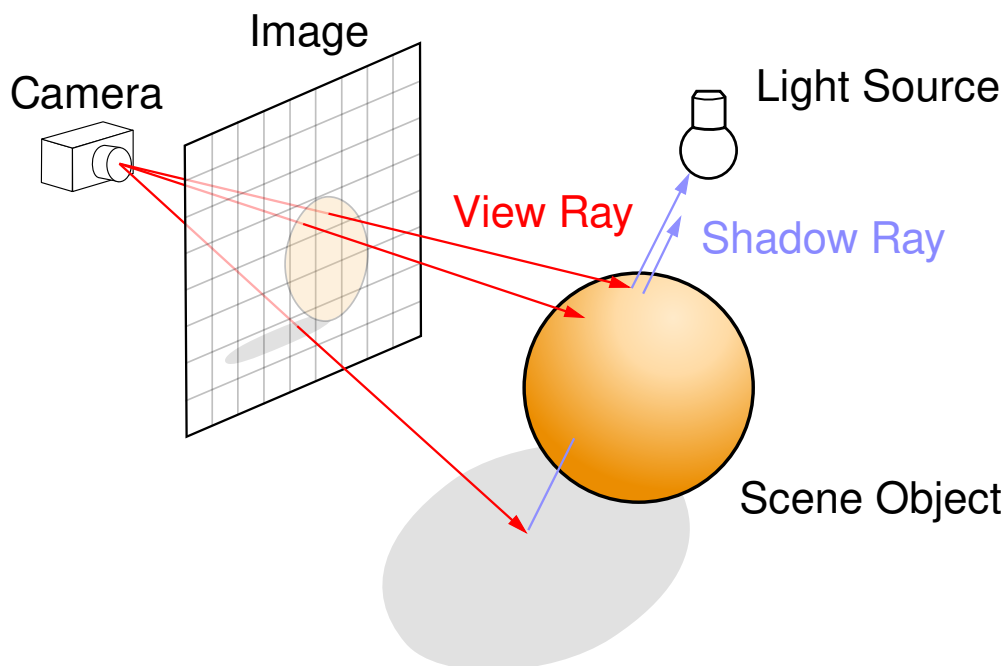


Рисунок 1.2 – Алгоритм обратной трассировки лучей

1.1.3 Выбор алгоритма

В качестве алгоритма рендеринга был выбран метод обратной трассировки лучей, так как для поставленной задачи не требуется полная физическая корректность модели, а также важна производительность, учитывая, что программа будет выполняться на центральном, а не графическом процессоре.

1.2 Анализ моделей освещения

Свет при взаимодействии с поверхностью может быть отражен, поглощен или преломлен. Часть световой энергии поглощается поверхностью и преобразуется в тепло. Отраженный и преломленный свет делают объект видимым для наблюдателя. Если весь падающий свет будет поглощен поверхностью объекта, он будет невидимым (абсолютное черное тело).

Все модели освещения делятся на два вида: глобальные и локальные. Глобальные модели учитывают возможности отражения и преломления света от объектов, не являющихся прямыми источниками освещения, поэтому они требуют значительных вычислительных затрат. Например, прозрачные поверхности должны позволять просматривать объекты за ними, на поверхностях возможно появление таких явлений, как рефлекс. Глобальные модели освещения основываются на том, что видимость и

затенения связаны между собой: яркость точки поверхности определяется распределением яркости по всем остальным поверхностям, видимым из этой точки. Так можно моделировать шероховатость и свойства отражения реальных материалов, освещение многократно отраженным светом и связанные с ним цветовые эффекты.

Существуют более простые, локальные модели освещения, которые учитывают только свет от источника. Они вычисляют интенсивность, цвет и дальнейшее распределение света на поверхности, но не учитывают перенос света между объектами. Изображение формируется в результате отражения, падающего на поверхность объектов света, интенсивность и цвет которого необходимо рассчитать. Фоновое освещение определяет цвет объекта в тени или при отсутствии явных источников освещения. Его интенсивность постоянна, а также равномерно распределена по всему пространству. Выделяют две основные модели локального освещения: модель Ламберта и модель Фонга.

1.2.1 Модель освещения Ламберта

Модель Ламберта описывает идеальную "матовую" или диффузно отражающую поверхность. Интенсивность отраженного света подчиняется закону косинусов Ламберта, согласно которому отраженное излучение одинаково во всех направлениях. При этом положение и угол обзора наблюдателя не имеют значения, т.к. диффузно отраженный свет рассеивается равномерно во всех направлениях.

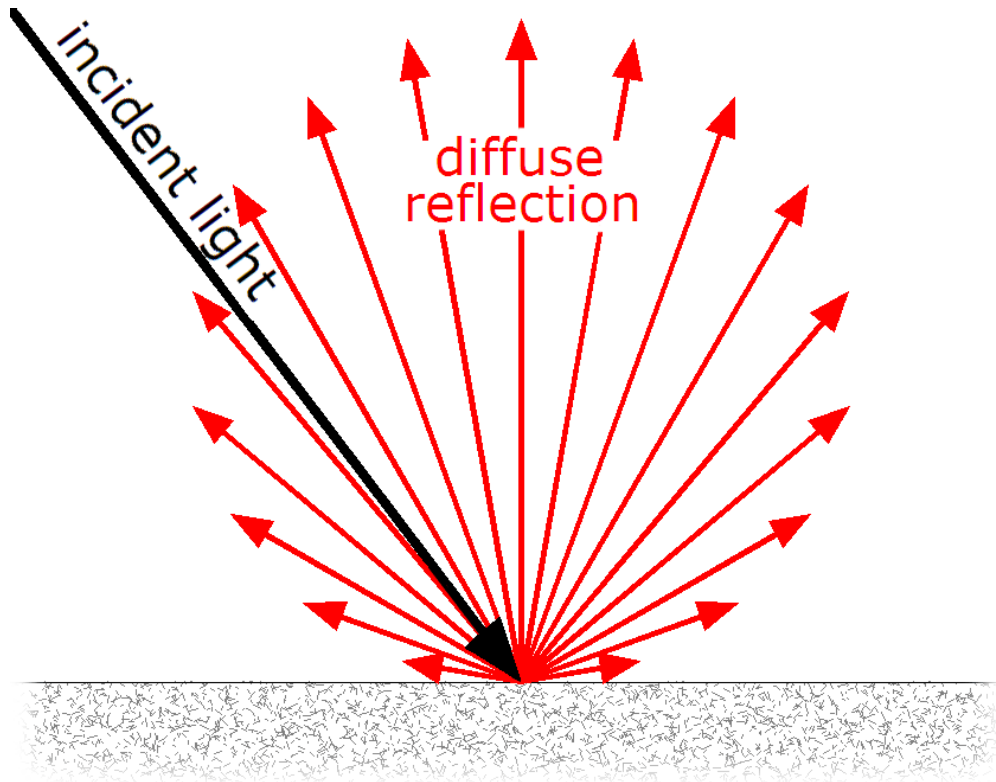


Рисунок 1.3 – Схема диффузного отражения Ламберта

Пусть:

- $\vec{N}$ — вектор нормали к поверхности;
- $\vec{L}$ — вектор от точки до источника;
- K_d — коэффициент диффузного освещения;
- I_0 — интенсивность источника;
- I — результирующая интенсивность света в точке.

Тогда формула расчета освещенности будет иметь следующий вид:

$$I = I_0 \cdot K_d \cdot (\vec{L} \cdot \vec{N}) \quad (1.1)$$

Из формулы (1.1) следует главный недостаток модели Ламберта - одинаковая интенсивность всех точек, принадлежащих одной грани.

Модель Ламбрета является одной из самых простых моделей освещения. Данная модель часто используется в составе других, более сложных,

моделей освещения, т.к. практически в любой другой модели можно выделить диффузную составляющую. Модель Ламберта представляется более-менее равномерной частью освещения и описывает ориентацию поверхностей сцены.

1.2.2 Модель освещения Фонга

Модель освещения Фонга представляет собой комбинацию диффузной составляющей (модель Ламберта) с зеркальной составляющей. Кроме равномерного освещения диффузного отражения, на объектах появляются блики за счет зеркального отражения. Падающий и отраженный лучи лежат в одной плоскости с нормалью к отражающей поверхности в точке падения, и эта нормаль делит угол на 2 равные части как показано на рисунке 1.4.

Зеркальная составляющая освещенности в точке зависит от того, насколько близко направления на наблюдателя и отраженного луча. Также в модели освещения Фонга используется фоновая составляющая, которая прибавляется к интенсивности в каждой точке. Она является грубым приближением лучей света, рассеянных соседними объектами, а затем достигшими заданной точки.

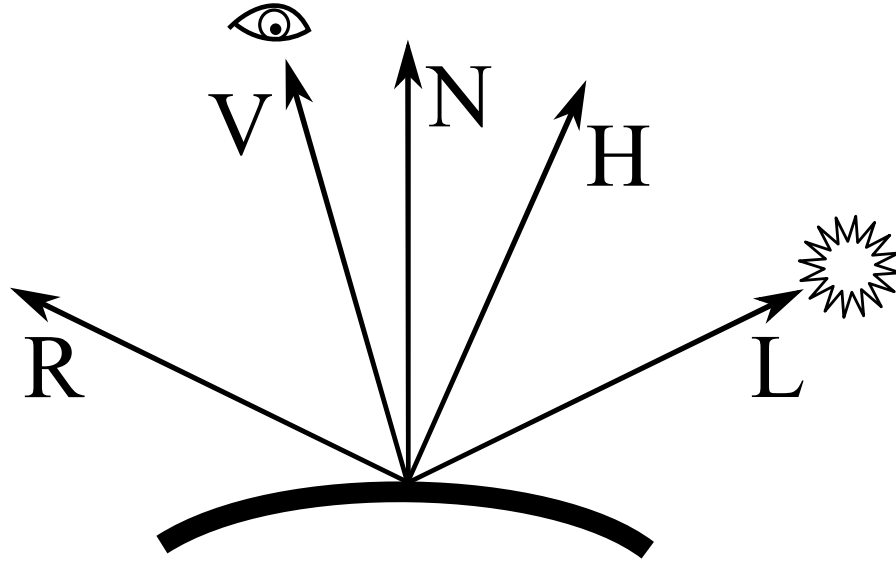


Рисунок 1.4 – Модель освещения Фонга, где

- $\vec{N}$ — вектор нормали к поверхности;
- $\vec{V}$ — вектор от точки до наблюдателя;
- $\vec{L}$ — вектор падающего света;
- $\vec{R}$ — вектор отраженного света;
- $\vec{H}$ — срединный вектор между вектором наблюдения и источником.

Пусть:

- K_s — коэффициент зеркального отражения;
- I_0 — интенсивность источника;
- α — коэффициент резкости блика;
- I_s — результирующая зеркальная составляющая света в точке.

Тогда зеркальная составляющая в модели Фонга рассчитывается следующим образом:

$$I_s = I_0 \cdot K_s \cdot (\vec{R} \cdot \vec{V})^\alpha \quad (1.2)$$

Согласно модели Фонга интенсивность в точке будет рассчитываться

по формуле:

$$I = I_a + I_d + I_s, \quad \text{где} \quad (1.3)$$

I_a — фоновая составляющая,

I_d — диффузная составляющая,

I_s — зеркальная составляющая

Подставив уравнение каждой из трех составляющих, получим следующую формулу:

$$I = I_0 \cdot K_a + I_0 \cdot K_d \cdot (\vec{L} \cdot \vec{N}) + I_0 \cdot K_s \cdot (\vec{R} \cdot \vec{V})^\alpha \quad (1.4)$$

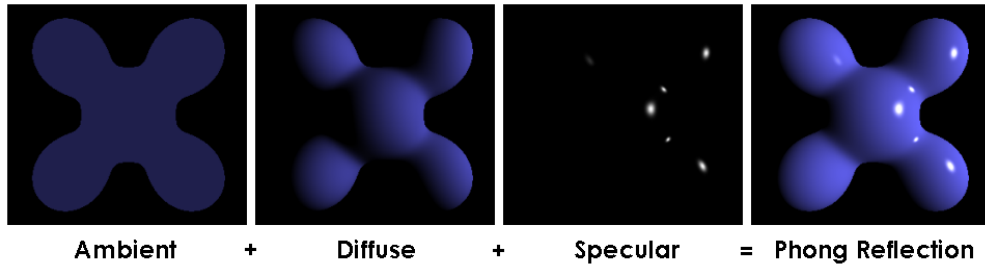


Рисунок 1.5 – Компоненты модели освещения Фонга

1.2.3 Модель освещения Блинна-Фонга

Модель освещения Блинна-Фонга является модификацией модели Фонга, в которой зеркальная составляющая рассчитывается следующим способом:

$$I_s = I_0 \cdot K_s \cdot (\vec{N} \cdot \vec{H})^\beta \quad (1.5)$$

Вектор $\vec{H}$ является нормализованной суммой векторов направления на источник $\vec{L}$ и направления на наблюдателя $\vec{V}$, его также называют полувектором.

$$\vec{H} = \frac{\vec{L} + \vec{V}}{\|\vec{L} + \vec{V}\|} \quad (1.6)$$

Основным преимуществом модификации Блинна-Фонга является лучшая производительность за счет упрощенного вычисления зеркальной составляющей, т.к. полувектор $\vec{H}$ вычисляется проще вектора отраженного света $\vec{R}$. Фоновая и диффузная составляющая данной модели рассчитыва-

ются аналогично модели Фонга.

1.2.4 Выбор модели освещения

В данной работе будет использована модель освещения Фонга, так как она максимально реалистично описывает диффузное и зеркальное отражение света.

1.3 Обзор геометрических примитивов

В данном разделе будут описаны способы задания рассматриваемых геометрических примитивов, а также способы пересечения луча с ними.

1.3.1 Сфера

Сфера определяется следующими параметрами:

- центром (C_x, C_y, C_z) ;
- радиусом r .

Каноническое уравнение сферы имеет вид:

$$(x - C_x)^2 + (y - C_y)^2 + (z - C_z)^2 = r^2$$

Уравнение луча в векторной форме:

$$\vec{P}(t) = \overrightarrow{origin} + \overrightarrow{direction} * t$$

...

1.3.2 Плоскость

1.3.3 Четырехугольник

1.3.4 Треугольник

1.3.5 Параллелепипед, правильная призма и пирамида

2 Конструкторский раздел

В этом разделе будут рассмотрены алгоритмы, выбранные для построения изображения, а также спроектированная диаграмма классов приложения.

2.1 Алгоритм построения изображений

В качестве средства описания алгоритма построения изображения будет использоваться диаграмма IDEF0.

Построение изображений в компьютерной графике в контексте диаграммы IDEF0 определяется следующими параметрами:

Описание входных данных

- Объекты определяются геометрией, положением и ориентацией и свойствами материала (цвет, текстуры, нормали);
- Камера определяется проекционными параметрами (перспективная, ортографическая), положением и ориентацией;
- Источники освещения определяются типом (направленный, точечный и др.) , положением и ориентацией (для направленных), цветом и интенсивностью.

Описание элементов управления

- Алгоритм рендеринга: растеризация, трассировка лучей, path tracing, ray marching;
- Модель освещения: локальная или глобальная;
- Способ затенения: простая, по Гуро, по Фонгу.

Описание механизмов

- Аппаратное обеспечение (ЦП, ОЗУ, ГП);
- Программное обеспечение;
- Язык программирования (C++, Python, C#, Java и др.);

- Графический API (OpenGL, Vulkan, DirectX, Metal и др.);
- Графическая библиотека/фреймворк (GLFW, SDL, SFML и др.).

Выходными данными является сгенерированный кадр.

Общий алгоритм генерация изображений представлен на рисунке 2.1.

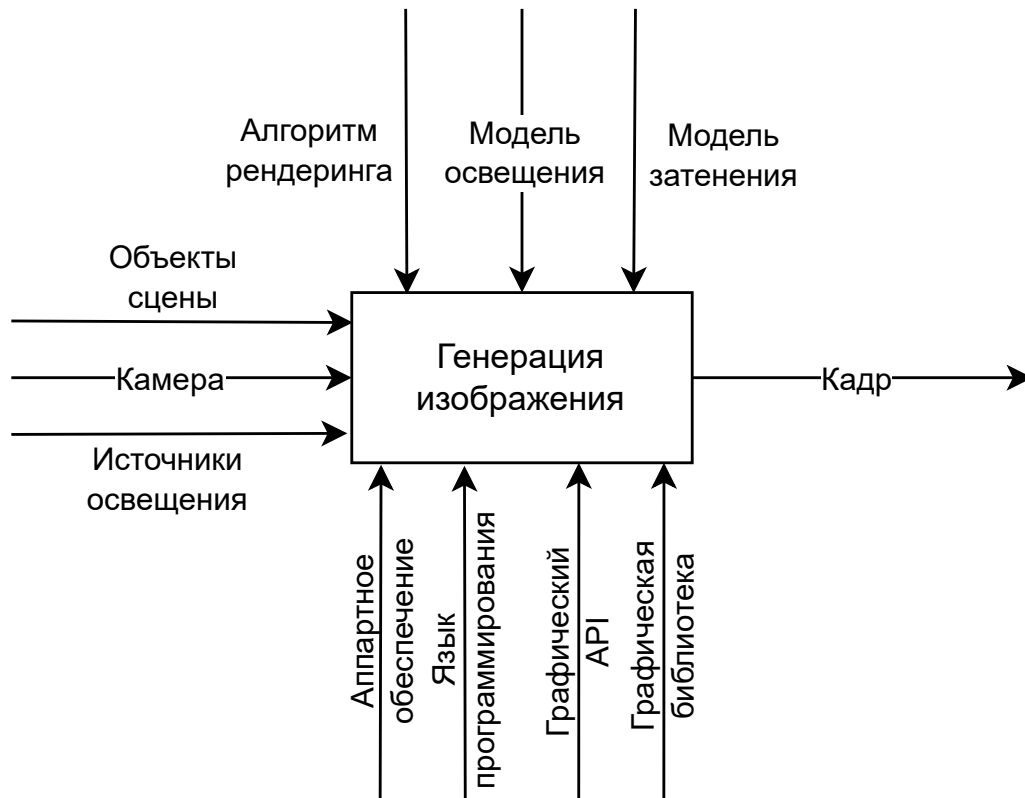


Рисунок 2.1 – IDEF0-диаграмма общего алгоритма построения изображения

Выбранный для реализации алгоритм построения изображений с конкретными методами, моделями и инструментами представлен на рисунке 2.2.

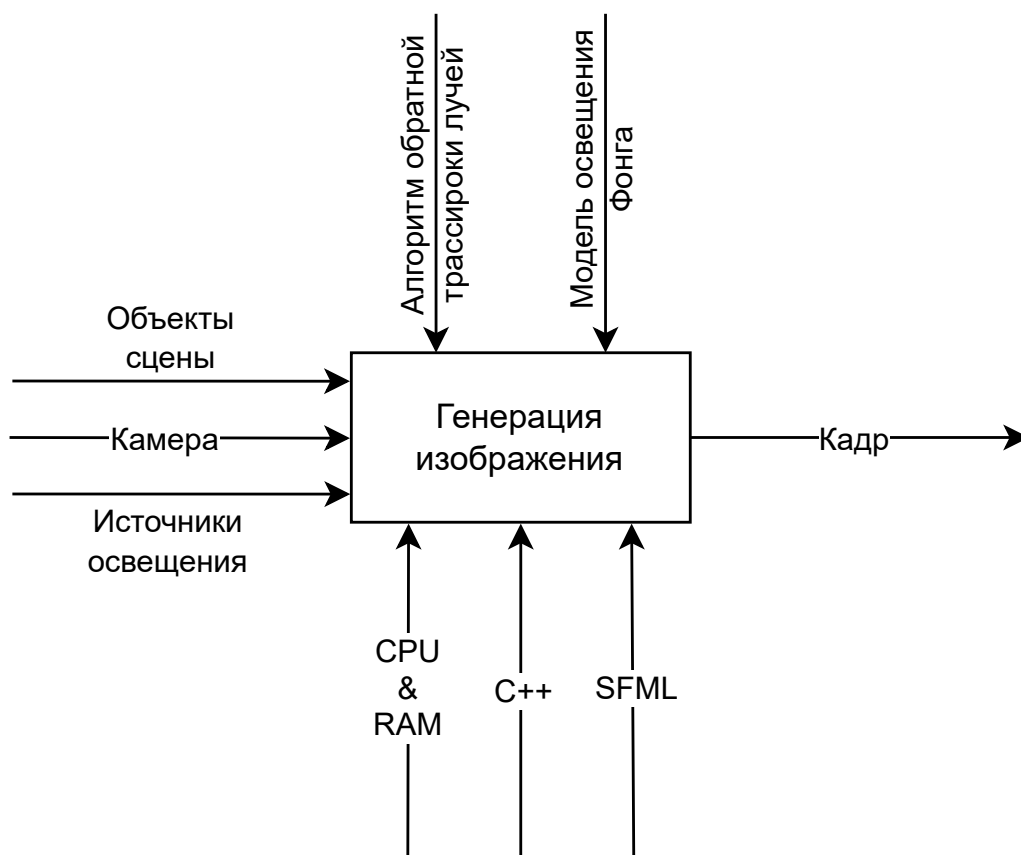


Рисунок 2.2 – IDEF0-диаграмма выбранного для реализации алгоритма построения изображения

2.2 Алгоритм обратной трассировки лучей

Схема алгоритма обратной трассировки лучей приведена на рисунке 2.3.

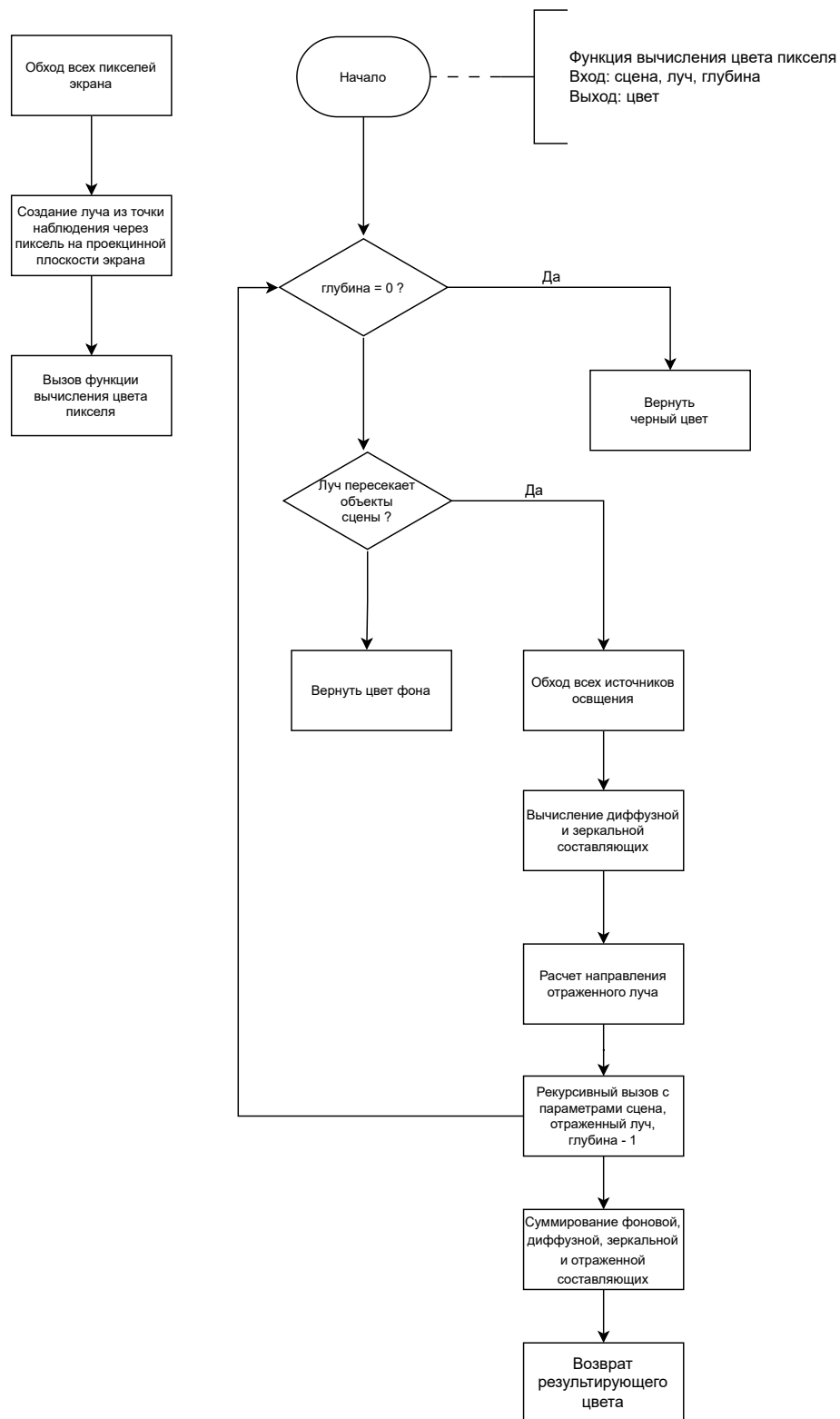


Рисунок 2.3 – Схема алгоритма обратной трассировки лучей

3 Технологический раздел

3.1 Требования к ПО

Разрабатываемое ПО должно обладать следующим функционалом:

- добавление и удаление объектов;
- изменение положения, ориентации, масштаба и цвета объектов;
- изменение положения и ориентации камеры;
- добавление и удаление источников освещения;
- изменение положения, ориентации и цвета источников освещения;

Разрабатываемое ПО должно соответствовать следующим требованиям:

- при запуске программы должна создаваться камера;

3.2 Средства реализации

Для написания данной курсовой работы был выбран C++. Выбор языка программирования обусловлен следующими факторами:

- быстроедействие;
- поддержка ООП;
- богатый выбор графических библиотек.

В качестве графической библиотеке была выбрана SFML. Выбор графической библиотеке был обусловлен следующими факторами:

- объектная модель (в отличие от GLFW, SDL).

Для реализация графического интерфейса пользователя был использован ImGui по причине легковесности, быстрогодействия и интеграции со всеми известными библиотекам. Для генерации файлов сборки проекта был использован CMake.

3.3 Диаграмма классов

Диаграмма классов в нотации UML приведена на рисунке 3.1.

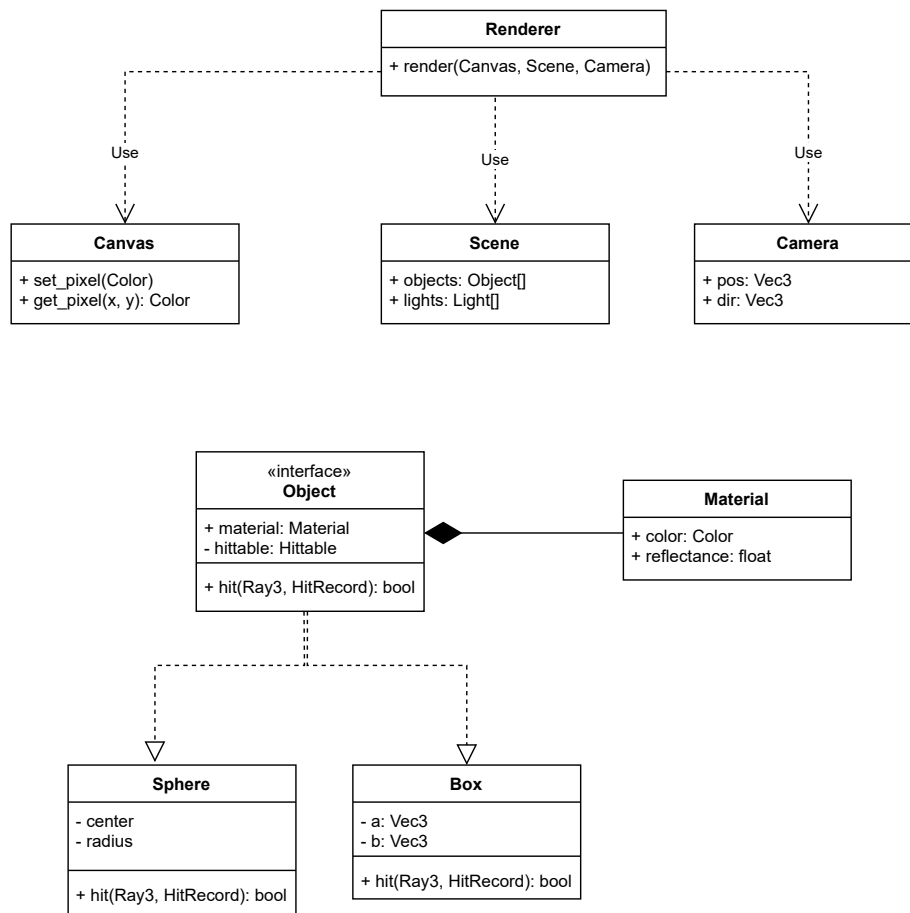


Рисунок 3.1 – Диаграмма классов приложения

3.4 Типы и структуры данных

Основные типы и структуры данных, используемые в программе.

- **Renderer**;
- **Scene**;
- **Camera**;
- **Object**.

3.5 Алгоритмы

Листинги программ нахождения пересечения луча с примитивами

3.6 Графический интерфейс пользователя

Несколько скриншотов и инструкция по использованию

4 Исследовательский раздел

4.1 Анализ производительности ПО

В качестве стандартной сцены для анализа времени рендеринга был выбран знаменитый чайник Юта (Utah Teapot).

Время рендеринга сцен для 1 потока

Число граней	Время генерации изображения (с)
108	1.408
532	5.306
1166	10.452
2148	19.490

Таблица 4.1 – Время генерация изображения 1 потоком в зависимости от числа граней

Время рендеринга сцен для 2 потока

Время рендеринга сцен для 4 потока

Время рендеринга сцен для 8 потока

Время рендеринга сцен для 16 потока

Выводы

При N-ом числе поток прирост производительности прекращается в связи с ...

ЗАКЛЮЧЕНИЕ

ПРИЛОЖЕНИЕ А